

A Key Schedule Design and Evaluation under Boundary Round-Key Leakage

Yu Morishima¹ Hideki Yoshikawa¹
Masahiro Kaminaga¹

¹Tohoku Gakuin University, Japan
morishima@mail.tohoku-gakuin.ac.jp

Abstract

We study key-schedule design under boundary round-key leakage, namely leakage of the first round key, the last round key, or both end round keys. We propose the nonlinear key-schedule $RK_i = K \oplus F(K \oplus T(i))$, where K is the master key, $T(i)$ is a public domain separation value, and F is a public SPN-based permutation parameterized by its round count N_F .

Under the boundary-leakage model considered in this paper, leakage of one end round key yields an instance of the equation $Z = U \oplus F(U)$, whereas leakage of both end round keys yields a differential constraint of the form $F(U) \oplus F(U \oplus \Delta) = \Gamma$, where Δ is determined by the two end indices and Γ is derived from the two leaked round-key values. These reductions clarify the nonlinear systems induced by boundary leakage and the absence of a linear elimination route to the master key.

We also evaluate reduced variants of the resulting systems through Gröbner basis experiments, and further examine them by SAT-based key-recovery experiments and right-censored runtime analysis via a Weibull AFT model. Within the tested range, we do not observe degree collapse or unusually strong linear bias. These results provide heuristic support for the view that, under the boundary-leakage model considered here, the tested instantiations of the proposed key-schedule family do not admit an obvious efficient inversion route.

Keywords: Key schedule, Round-key leakage, Side-channel attacks, SAT-based cryptanalysis

1 Introduction

Modern block ciphers are usually designed and analyzed with most of the attention paid to the round function [1]. The structure of key schedules, however, is often treated as a secondary issue, as in classical designs such as DES [2, 1]. Nevertheless, secure and efficient key schedules are crucial in constrained hardware, especially when resistance against side-channel and fault attacks is required [3, 4, 5, 6, 7, 8].

In several block ciphers, key schedules have historically been designed to be simple, often consisting of rotations, xor with round constants, and a few local updates. Such almost linear structures are easy to implement, but they may offer weak protection once partial key-schedule material is exposed; in many block cipher designs, the master key can be reconstructed from one or two leaked round keys or from leaked fragments of the expanded key or key schedule [9, 10, 11]. Despite this, comparatively little work explicitly targets key schedules that remain robust under such round-key leakage. In particular, key-schedule leakage offers only one or two instances under a fixed secret, so security evaluation differs fundamentally from both chosen-plaintext and known-plaintext attack settings typically considered for block ciphers.

Accordingly, classical statistical distinguishers that amplify small biases over many samples (e.g., differential or linear cryptanalysis) are not the right abstraction here; instead, side-channel and fault attacks may reveal partial or even complete round keys, or other key-schedule material, while leaving the master key itself hidden [3, 4, 5, 6, 7, 8, 12, 13]. This makes the key schedule a direct object of inversion under leakage and naturally leads to solving a small number of nonlinear relations, as in algebraic cryptanalysis and equation-solving approaches such as Gröbner bases and SAT [14].

Prior work on key-schedule security has mostly followed three lines. First, weak schedules have been exploited in related-key and key-schedule cryptanalysis, and more generally through slide attacks that capitalize on round-to-round self-similarity [11, 15]. Second, the AES line of work showed that slow diffusion and local collisions in the key schedule can become decisive in strong related-key settings, including attacks on full AES-192 and AES-256 [16]. Third, the key expansion itself has been treated as a practical attack surface in physical cryptanalysis, for example by differential fault analysis and simple power analysis on the AES key schedule [17, 18]. There is also work on modifying AES-like schedules to strengthen resistance against related-key differential and boomerang attacks [19]. In contrast, our aim is not to harden an existing standard cipher against a broad chosen-related-key model, but to study whether an explicit nonlinear key schedule can itself impede algebraic inversion when only one or two round keys leak under a fixed unknown master key.

The aim of this paper is to propose and study an explicit nonlinear key-schedule intended to resist algebraic inversion when one or two round keys leak, without resorting to an additional pseudo random function (PRF) primitive. This choice lets us evaluate whether robustness can arise from the key-schedule structure itself, rather than from a separate keyed primitive. The design uses a nonlinear layer given by a substitution-permutation network (SPN) and generates each round key through an explicit nonlinear masking of the master key.

Given an n -bit master key K , a round index i , and a public domain separation value $T(i)$ derived from i , we define the i -th round key RK_i by

$$RK_i(K) := K \oplus F(K \oplus T(i)),$$

where F is a N_F -round n -bit SPN. The function F is built from a 4-bit S-box, a nibble-wise maximum distance separable (MDS) diffusion layer, and a permutation of the nibbles; details are given in Section 3. The main contributions of this work are as follows.

(i) *Leakage model and explicit nonlinear key-schedule design.* We formulate a round-key leakage model tailored to key schedules, where the adversary cannot choose the key-schedule input and obtains either the first round key, the last round key, or both end round keys for a fixed but unknown secret. Under this model, the proposed construction uses a public domain-separation value $T(i)$ and an SPN-based public permutation F , so that K appears both outside and inside the nonlinear layer. As a result, a leaked round key does not yield a direct linear constraint on K .

(ii) *Heuristic structural and algebraic evaluation under boundary round-key leakage.* We derive the nonlinear relations induced by one or two leaked round keys and assess whether they admit an evident efficient inversion route to the master key or to additional round keys. The evaluation combines structural analysis with reduced-size Gröbner basis experiments and SAT-based solving experiments, and the results are interpreted as heuristic support for the absence of an obvious efficient inversion route in the tested settings.

2 Threat Model

2.1 Key-Schedule Structure under Round-Key Leakage

In its evaluation of AES round 1 candidates, NIST examined whether exposure of a small number of subkeys or round keys could enable recovery of the master key [20]. The round 1 status report summarizes this aspect by classifying candidates according to whether master-key recovery appears feasible from only a few subkeys, as opposed to requiring analysis of a substantially larger portion of the key expansion [20]. Motivated by this discussion, we focus on the minimal nontrivial leakage setting, namely one or two leaked round keys, which captures whether a key schedule exhibits an immediate inversion route from a very small number of leakage instances.

In that discussion, several candidates (including Rijndael, Serpent, and Twofish) were described as more vulnerable in the sense that the master key can be derived from a small set of subkeys, whereas others (including RC6 and MARS) were described as less vulnerable since master-key recovery would appear to require attacking many rounds [20]. In this paper, RC6 and similar candidates serve only as reference points suggesting that obvious local invertibility from one or two leaked round keys need not be inevitable, rather than as quantitative baselines for later comparison.

At the same time, the NIST report does not pinpoint which structural features account for this difference, nor does it formalize the issue under an explicit leakage model. In particular, it does not attribute the reported resistance to any single mechanism such as stateful iterative expansion or data-dependent operations.

A standard design option for mitigating round-key leakage is to derive each round key from the master key using a dedicated PRF. Adopting such an additional keyed primitive, however, changes the object of analysis. Under the usual black-box idealization, leakage of one or two round keys reveals only a constant number of outputs of that primitive. Consequently, the security argument rests primarily on the assumptions and leakage behavior of that primitive, rather than on the structure of the key schedule itself.

Our objective is therefore to study the question directly. Under a concrete and

motivated boundary-leakage model—namely leakage of the first round key, the last round key, or both end round keys—when does the structure of a key schedule admit efficient recovery of the master key? Accordingly, we focus on robustness that arises from the key-schedule construction itself, without placing the main burden of the analysis on an additional keyed primitive.

2.2 Boundary Round–Key Leakage Model

The round–key leakage model considered in this work is motivated by practical side-channel and fault attacks on real block ciphers, in which round subkeys or other expanded-key material may be exposed while the master key itself remains secret [3, 4, 5, 6, 7, 8, 12]. We consider leakage of either a single end round key RK_i or a pair of end round keys (RK_i, RK_j) taken from the two ends of the schedule, where i and j correspond to the first and last round indices, respectively. This abstract boundary-leakage model covers, for example, a one-round reduced cipher with key mixing at the input and output boundaries, as well as implementation-level settings in which leakage is concentrated near the beginning or the end of the execution trace, where round-key additions are directly exposed through the implementation. We now formalize the leakage setting considered in this paper. Let n denote the master–key and state size in bits.

Definition 1 (Round–key leakage model) *Let $K \in \{0, 1\}^n$ be the master key, and let RK_i denote the round key for round i , where i ranges over a finite index set. The adversary is assumed to have full knowledge of the specification of the block cipher and of the key schedule, except for K . In the present model, leakage is restricted to boundary round keys. More precisely, the adversary may learn either one end round key RK_i or both end round keys (RK_i, RK_j) , where i and j are the first and last round indices, respectively. The leakage is assumed to be exact, and the adversary obtains the full n -bit round-key value without errors. We do not assume chosen plaintext or chosen ciphertext oracle access. The analysis considers only the algebraic relations induced by leaked round keys.*

In this paper, we do not model arbitrary leakage positions across all round indices. Instead, we focus on a practically motivated boundary-leakage setting and consider only the induced algebraic relations without chosen-plaintext or chosen-ciphertext access. In the experiments, we instantiate one-key leakage by the canonical representative $i = 0$, and two-key leakage by the pair $(i, j) = (0, 1)$, which in the present one round instantiation is exactly the pair of the two end round keys. Thus, the main design parameter under study is the internal SPN depth N_F of the round-key generation map F , rather than variation across leakage positions.

Remark 1 (Security objective under round–key leakage)

Under the model of Definition 1, an adversary aims either (i) to recover the master key K , or (ii) to predict a round key RK_k for some index k that was not leaked. Informally, the intended goal is that no algorithm running in practical time should recover K with success probability significantly larger than 2^{-n} , namely the baseline success

probability of random guessing over the n -bit key space. In particular, leakage of one or two round keys should not provide a meaningful practical advantage over exhaustive search on K , nor should it enable efficient prediction of any unobserved round key.

The later sections address only the question of whether leakage of one or two round keys yields relations from which the master key or additional round keys can be recovered efficiently. Their conclusions are structural and heuristic, and should not be read as a general proof of leakage resilience. In particular, we do not claim security under arbitrary leakage positions, broader leakage settings, or different implementation assumptions.

3 Proposed Key Schedule

Under the leakage model of Section 2, our objective is to study an 128-bit key-schedule parameterized by the SPN round count N_F , whose robustness is intended to derive from the schedule structure itself, without relying on a PRF primitive. In particular, leakage of one or two round keys should not lead to an evident efficient inversion route to the master key.

To this end, we generate each round key directly from the master key and a public domain separation value through a public permutation F . A purely inner form such as

$$\text{RK}_i = F(K \oplus T(i))$$

would be immediately invertible from a single leaked round key because F is public and bijective. We therefore include an outer xor with K and define

$$\text{RK}_i(K) := K \oplus F(K \oplus T(i)). \quad (1)$$

Here $T(i) \in \{0, 1\}^{128}$ is a public value that depends only on the round index i and provides domain separation across rounds. The function $F : \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}$ is a public permutation instantiated as an N_F -round SPN described in Section 3.2. All internal values are represented as 32 nibbles. In our experiments, we instantiate $N_F = r$ with $r \in \{1, 2, 3, 4, 5\}$. The notation $\oplus$ denotes bitwise xor.

The remaining subsections specify the public domain separation value $T(i)$ and the SPN permutation F .

3.1 Domain Separation Value $T(i)$

Let the key schedule generate N round keys, with round indices $i \in \{0, 1, \dots, N-1\}$, and assume $N \leq 2^{32}$. For each round index i we derive a 128-bit value $T(i)$ that serves as a domain separation value.

We regard i as a 32-bit word and embed it into 128 bits by

$$\tilde{i} := i \parallel i \parallel i \parallel i \in \{0, 1\}^{128}.$$

We define

$$\begin{aligned} T(i) &= \text{ROTL}_{32}(\tilde{i}, r_0) \oplus \text{ROTL}_{32}(\tilde{i}, r_1) \\ &\quad \oplus \text{ROTL}_{32}(\tilde{i}, r_2) \oplus c, \end{aligned}$$

where ROTL_{32} denotes rotation by a fixed number of bits within each 32-bit word, the rotation amounts r_0, r_1, r_2 are public constants with pairwise distinct values, and c is a fixed public 128-bit constant. In our reference implementation we set

$$(r_0, r_1, r_2) = (5, 13, 29),$$

$$c = 0x000102030405060708090a0b0c0d0e0f.$$

The computation of $T(i)$ uses only 32-bit rotations and xors. It is intended to provide domain separation across rounds by making $T(i)$ distinct for different indices i , and hence making the inputs $T(i) \oplus K$ to the nonlinear layer F distinct for fixed K . The following proposition formalizes this property.

Proposition 1 *Assume $N \leq 2^{32}$. Then the map $T : \{0, \dots, N-1\} \rightarrow \{0, 1\}^{128}$ is injective. In particular, $T(i) \neq T(j)$ whenever $i \neq j$.*

Proof. Consider the induced $\text{GF}(2)$ -linear map on 32-bit words,

$$\Phi(u) := \text{ROTL}(u, r_0) \oplus \text{ROTL}(u, r_1) \oplus \text{ROTL}(u, r_2),$$

where ROTL denotes rotation of a single 32-bit word, and r_0, r_1, r_2 are distinct public constants. Identifying 32-bit words with $\text{GF}(2)[x]/(x^{32} - 1)$, the map Φ corresponds to multiplication by

$$p(x) = x^{r_0} + x^{r_1} + x^{r_2}.$$

Over $\text{GF}(2)$ we have $x^{32} - 1 = (x - 1)^{32}$. Since $p(1) = 1 + 1 + 1 = 1 \neq 0$, the factor $(x - 1)$ does not divide $p(x)$. Hence $\gcd(p(x), x^{32} - 1) = 1$, and Φ is invertible.

Now suppose $T(i) = T(j)$. Since xor with the fixed constant c does not affect equality, this implies $\Phi(i) = \Phi(j)$ as 32-bit words, hence $i = j$ by invertibility of Φ . Therefore T is injective on $\{0, \dots, N-1\}$ for $N \leq 2^{32}$.

3.2 Structure of the SPN F

The function F is the 128-bit SPN used in the main construction, with N_F rounds. Our notion of a SPN follows the design philosophy of PRESENT [21]. Each round consists of an S-box layer, a columnwise diffusion layer, a nibble permutation, and the addition of a round constant.

Let S be the parallel application of the PRESENT S-box to all 32 nibbles of the state. Let L_M denote the columnwise application of a 4×4 matrix M over $\text{GF}(2^4)$, and let π be the nibble permutation. For a state $X = (x_j)_{j=0}^{31} \in (\{0, 1\}^4)^{32}$, the induced action of π is defined by $Y_{\pi(j)} = x_j$ for all j , where $Y = \pi(X)$. Let $C_1, \dots, C_{N_F}$ be fixed 128-bit round constants. For $t \in \{1, \dots, N_F\}$ define

$$R_t(X) := \pi(L_M(S(X))) \oplus C_t, \tag{2}$$

and set

$$F(X) := R_{N_F} \circ \dots \circ R_2 \circ R_1(X). \tag{3}$$

S-box layer. Each nibble is replaced by the four-bit PRESENT S-box. This S-box is well studied and is efficient to implement in hardware and software.

Diffusion layer. We view the 128-bit state as a 4×8 array of nibbles. Under the indexing $j = 8r + c$, where $r \in \{0, 1, 2, 3\}$ and $c \in \{0, \dots, 7\}$, column c consists of nibble positions $c, 8 + c, 16 + c, 24 + c$, and in each column we apply a 4×4 matrix over $\text{GF}(2^4)$. The field $\text{GF}(2^4)$ is defined by the irreducible polynomial $x^4 + x + 1$. We use

$$M = \begin{bmatrix} 4 & 1 & 2 & 2 \\ 8 & 6 & 5 & 6 \\ 11 & 14 & 10 & 9 \\ 2 & 2 & 15 & 11 \end{bmatrix},$$

where each entry is a nibble interpreted as an element of $\text{GF}(2^4)$ and the field multiplication is performed modulo $x^4 + x + 1$.

We choose the matrix M to provide strong diffusion within each column. In particular, we aim for an invertible linear layer with maximum branch number, so that any nonzero input difference activates multiple output nibbles, as quantified by the bound $\text{wt}(a) + \text{wt}(Ma) \geq 5$. The following lemma formalizes these properties of M , where $\text{wt}(\cdot)$ denotes the Hamming weight over $(\text{GF}(2^4))^4$, namely, the number of nonzero components.

Lemma 2 (Invertibility and MDS property of M) *The matrix M is invertible over $\text{GF}(2^4)$. Moreover, its branch number*

$$B(M) := \min_{0 \neq a \in (\text{GF}(2^4))^4} (\text{wt}(a) + \text{wt}(Ma))$$

equals 5, which is the maximum possible value for a 4×4 matrix over $\text{GF}(2^4)$.

Proof. Direct computation shows that $\det(M) \neq 0$ in $\text{GF}(2^4)$, hence M is invertible. We verified $B(M) = 5$ by exhaustive search over all $2^{16} - 1$ nonzero inputs $a \in (\text{GF}(2^4))^4$.

Permutation of nibbles. We view the state as a 4×8 array of nibbles, indexed by rows $r \in \{0, 1, 2, 3\}$ and columns $c \in \{0, \dots, 7\}$.

Each nibble position is indexed by

$$j = 8r + c.$$

Equivalently, for a state $X = (x_j)_{j=0}^{31}$ and $Y = \pi(X)$, we set $Y_{\pi(8r+c)} = x_{8r+c}$. We implement the nibble permutation as cyclic shifts within each row, and define π by

$$\begin{aligned} \pi(8r + c) &= 8r + (c + s_r \bmod 8), \\ (s_0, s_1, s_2, s_3) &= (1, 3, 5, 7). \end{aligned} \tag{4}$$

For the 128-bit specification, the permutation π is designed to reduce column locality. After one round, the four nibbles of any column are distributed across four distinct

columns, and, when combined with the columnwise diffusion layer, this promotes inter-column diffusion across rounds. The following lemma summarizes the basic properties of π needed for this argument.

Lemma 3 (Properties of π for the 128-bit specification)

The map π is a bijection on the set of 32 nibble positions. Moreover, for any fixed column, the four nibbles in that column are mapped to four distinct columns.

Proof. For the 128-bit specification, the state is viewed as a 4×8 array of nibbles. For each fixed row r , the map

$$c \mapsto c + s_r \bmod 8$$

is a bijection on the set $\{0, \dots, 7\}$. Therefore, the permutation π defined in (4) is bijective, with inverse obtained by subtracting s_r modulo 8 on each row. Now fix a column index c . The four nibbles at positions (r, c) for $r \in \{0, 1, 2, 3\}$ are mapped to columns with indices

$$c + s_r \bmod 8.$$

Since the values 1, 3, 5, 7 are pairwise distinct modulo 8, these four column indices are distinct.

Round constants. At each round, a 128-bit constant is added by xor. In the specification, the round constants are generated from a 128-bit state x by applying the following three-step xorshift update successively:

$$x \leftarrow x \oplus (x \ll 13), x \leftarrow x \oplus (x \gg 7), \quad x \leftarrow x \oplus (x \ll 17),$$

where $\ll$ and $\gg$ denote left and right shifts on 128-bit words, respectively, and bits shifted out are discarded.

In the reference implementation, we initialize the recurrence with $x_0 = 0x6a09e667f3bcc909bb67ae8584caa73b$, and let x_t be the state obtained after applying the above three-step update t times. We set $C_t := x_t$ for $t = 1, \dots, N_F$.

With these components, we state two basic properties of the SPN F . The first records that F is a permutation. The second quantifies two round diffusion by a lower bound on the number of active S-boxes, which we record for reference.

Proposition 4 (F is a permutation) *The function $F : \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}$ defined in (3) is a permutation.*

Proof. The substitution layer S is a permutation on the set of $n/4$ nibbles by definition of the PRESENT 4-bit S-box. The diffusion layer L_M is a permutation since the 4×4 matrix M is invertible over $\text{GF}(2^4)$ by Lemma 2. The nibble permutation π is a bijection on the $n/4$ nibble positions by Lemma 3. Finally, xor with a fixed round constant is a permutation on $\{0, 1\}^n$. Therefore, each round function R_t defined in (2) is a permutation, and so is their composition F .

Proposition 5 (Two round lower bound on active S-boxes) *Consider two consecutive rounds R_t and R_{t+1} of F for some $1 \leq t < N_F$. For any nonzero input nibble difference, the total number of active S-boxes in these two rounds is at least 5.*

Proof. Let the input difference activate at least one nibble in some column, and let $a \in (\text{GF}(2^4))^4$ be the 4-nibble difference vector in that column after the first S-box layer (so $a \neq 0$). By Lemma 2, the MDS layer satisfies $\text{wt}(a) + \text{wt}(Ma) \geq B(M) = 5$. After applying π , the $\text{wt}(Ma)$ active nibbles are moved into $\text{wt}(Ma)$ distinct columns by Lemma 3, hence they correspond to $\text{wt}(Ma)$ active S-boxes in the next round. Therefore the total number of active S-boxes over these two rounds is at least $\text{wt}(a) + \text{wt}(Ma) \geq 5$.

4 Structural Analysis Under Round-Key Leakage

In this section we study the security of the proposed key schedule under the round-key leakage model. Define $U_i := K \oplus T(i)$, so that $\text{RK}_i = K \oplus F(U_i)$. We consider the cases where one or two round keys leak, and discuss the difficulty of predicting other round keys. Our analysis is heuristic: it formalizes structural obstacles suggested by the design, but it is not a proof of indistinguishability, and it treats the key schedule relations in isolation from a specific round function.

4.1 Leakage-Induced Constraints and Reduction to F -Inversion

For the proposed schedule, each leakage instance can be rewritten in an equivalent algebraic form. For a leaked index i , define

$$U_i := K \oplus T(i), \quad Z_i := \text{RK}_i \oplus T(i).$$

Then the leakage equation becomes

$$Z_i = U_i \oplus F(U_i).$$

Thus, leakage of a single round key yields one nonlinear equation in a single unknown 128-bit value U_i . Once U_i is recovered, the master key follows as $K = U_i \oplus T(i)$.

Remark 2 *As shown above, leakage of a single round key reveals exactly one equation of the form $Z = U \oplus F(U)$ in an unknown U . This rewriting is purely algebraic and does not require access to an F^{-1} oracle. Accordingly, the one-round-key leakage recovery problem reduces to the difficulty of inverting the mapping $G_F(U) = U \oplus F(U)$ for a single instance. Since only one leakage instance is available for a fixed but unknown secret, techniques that rely on many independent samples do not apply. This motivates evaluating the practical solvability of the resulting single-instance inversion problem using equation-solving techniques.*

If two distinct indices $i \neq j$ leak, let

$$\Delta := T(i) \oplus T(j).$$

Since $U_j = K \oplus T(j) = U_i \oplus \Delta$, combining the two leakage equations yields the differential constraint

$$F(U_i) \oplus F(U_i \oplus \Delta) = Z_i \oplus Z_j \oplus \Delta.$$

Hence, the two-leakage setting reduces to solving for a single unknown 128-bit value U_i from one differential relation on the public permutation F with known input difference Δ and known output difference $Z_i \oplus Z_j \oplus \Delta$.

Equivalently, taking the xor $RK_i \oplus RK_j$ cancels the outer xor with K and yields $F(U_i) \oplus F(U_j)$ with $U_j = U_i \oplus \Delta$. However, both inputs remain keyed, so this relation does not simplify to linear constraints in the key bits without additional structure.

In summary, under one or two leakage instances, the adversary obtains only a small number of nonlinear constraints in a single unknown 128-bit value.

To make this algebraic structure more explicit in the one-leakage setting, we record the following basic degree upper bound. It shows that each output bit of the induced round-key equation inherits the round-dependent algebraic growth of the SPN F , and therefore does not a priori reduce to a trivially low-degree expression merely because only one round key leaks.

Proposition 6 (Degree upper bound for one-leakage) *Let F be instantiated as an N_F -round SPN using a 4-bit S-box of algebraic degree d_S . For the 128-bit specification, each output bit of $RK_i(K)$, viewed as a Boolean polynomial in the 128 key bits of K , has algebraic degree at most $d_S^{N_F}$.*

Proof. For the 128-bit specification, the key schedule Eq. (1) corresponds to 128 Boolean equations in the 128 key bits of K . Each round of F consists of an S-box layer followed by an affine transformation, which does not increase algebraic degree. Hence each output bit of $F(X)$ has degree at most $d_S^{N_F}$ in the input bits of X . The outer xor with K does not increase algebraic degree, proving the claim.

4.2 Prediction of Other Round Keys

Another natural goal for the adversary is to predict an unobserved round key RK_k for some index k that does not leak. Such a round key is given by

$$RK_k = K \oplus F(K \oplus T(k)).$$

Since T is injective, the unknown input $U_k := K \oplus T(k)$ differs from the unknown inputs corresponding to leaked indices. The relations obtained from one or two leaked round keys constrain only those leaked-index inputs and do not directly yield $F(U_k)$.

Within the present analysis, predicting an unobserved round key RK_k from one or two leaked round keys therefore appears to require solving the same underlying recovery problem for K or otherwise evaluating F on U_k .

5 Experimental Evaluation

This section presents an experimental evaluation under the leakage model of Definition 1. We evaluate the applicability of equation-solving attacks to the leakage-induced

systems when only one or two round keys leak. The proposed construction itself is defined for 128-bit master keys. Since direct experimental evaluation at the full 128-bit size becomes expensive for larger SPN round counts, we additionally consider reduced-size exploratory variants with state size $n \in \{16, 32, \dots, 128\}$. These variants should be viewed as a truncated exploratory family, not as faithful structural proxies for the 128-bit specification. For each such n , we define an n -bit SPN F_n on a $4 \times (n/16)$ nibble array: the PRESENT S-box and the columnwise application of M are unchanged, the row-shift permutation is $\pi_n(r, c) = (r, c + s_r \bmod (n/16))$ with $(s_0, s_1, s_2, s_3) = (1, 3, 5, 7)$, and the domain separation values $T_n(i)$ and round constants are obtained by taking the low n bits of $T(i)$ and C_t , respectively. In particular, the four-distinct-columns property of the 128-bit permutation is preserved only when the residues of $(1, 3, 5, 7)$ modulo $n/16$ remain pairwise distinct. Within the tested grid, this holds for $n \in \{80, 112, 128\}$ and fails for $n \in \{16, 32, 48, 64, 96\}$. Accordingly, the reduced-size experiments below are used only to study solver behavior on the family F_n , not as stand-alone structural evidence for the full 128-bit design. We report empirical recovery rates, the number of correct master key bits in returned assignments, and time-to-solve statistics.

5.1 Evaluation scope and modeling

From one or two leaked round keys, an adversary derives a small set of nonlinear relations as described in Section 4. We treat recovery of the unknown value U_i , and hence of K , as the primary solving target.

Our goal here is not to provide a reduction-style proof, but to assess empirically to what extent equation-solving attacks appear applicable to the leakage-induced systems considered in this paper. For dense systems arising from only one or two leakage instances, such methods offer limited general performance guarantees, so the present evaluation is intentionally heuristic.

5.2 Experimental Setup

For each parameter pair (n, r) , we sampled 50 independent random master keys $K \in \{0, 1\}^n$ and generated the corresponding one-key and two-key leakage instances. For the reduced-size variants, we varied the state size as $n \in \{16, 32, 48, \dots, 128\}$ and varied the number of SPN rounds r in the public permutation F_n as $r \in \{1, 2, 3, 4, 5\}$.

The main object of study remains the full 128-bit specification. We nevertheless include reduced-size exploratory variants because, under the present 600-second budget, the larger-size regime becomes heavily right-censored already at small round counts, while for smaller n the permutation layer π acts on only $n/16$ columns so the row shifts $(1, 3, 5, 7)$ may collide modulo the reduced number of columns. Hence these reduced-size variants are not treated as faithful structural proxies; they are used only to expose solver-side runtime behavior in a less heavily censored regime.

We used a fixed wall-clock timeout of 600 seconds per run and recorded the observed time to solve. All experiments were executed on machines equipped with an Intel Core i9-13900K CPU and 64 GB RAM. All solver runs were restricted to a single performance core. We report per-core results. Throughout the experiments, the one-key setting used the leaked round-key index $i = 0$, whereas the two-key setting used the leaked round-key

pair $(i, j) = (0, 1)$. In the present one round setting, this pair is exactly the pair of the two end round keys in the boundary-leakage model of Section 2. Since the principal design parameter studied here is the internal SPN round count $r = N_F$ of the round-key generation map F_n , the experiments are intended to isolate how increasing this internal nonlinear mixing changes the difficulty of solving the induced leakage equations under representative boundary leakage. We do not attempt a systematic comparison across leakage positions.

5.3 SAT-based experiments

5.3.1 Key recovery rates vs (n, r)

Following the setup in Section 5.2, we evaluate SAT-based key recovery under one-key and two-key leakage using CaDiCaL 1.9.5. For each parameter pair (n, r) , we aggregate results over 250 solver runs, corresponding to 50 generated SAT instances with 5 random seeds per instance.

For reporting, we use three recovery metrics. Let $\hat{K}$ denote the key returned by the solver in a non-timeout run, and let $C(\hat{K}, K)$ denote the number of correctly recovered key bits. Full denotes the percentage of solver runs for which $C(\hat{K}, K) = n$, i.e., exact master-key recovery. Partial denotes the percentage of solver runs for which $C(\hat{K}, K) > n/2$, i.e., the returned assignment contains more correct than incorrect key bits. Finally, μ denotes the mean of $C(\hat{K}, K)$ over the non-timeout runs.

The threshold $n/2$ is used as a natural random-guess baseline: for an uninformed random n -bit guess, the expected number of matching bits is $n/2$. We therefore use $C(\hat{K}, K) > n/2$ as a coarse descriptive criterion for majority-correct recovery at the bit level, rather than as a statistical significance test. Since exact recovery implies majority-correct recovery, Full is a subset of Partial and the two metrics are not mutually exclusive.

Tables 1 and 2 summarize the results for the one-key and two-key settings, respectively. Here, TO indicates that all solver runs for the corresponding parameter pair timed out. The smallest tested sizes, especially $n = 16$ and $n = 32$, should therefore be read only as exploratory reduced instances. In these settings, and more generally whenever the reduced number of columns causes collisions among the row shifts of π , the induced family no longer preserves the full column-separation behavior of the 128-bit specification. These points are included because the larger-size regime becomes timeout-dominated too early to expose the transition into hard instances from raw recovery tables alone. Accordingly, the observed dependence on (n, r) is descriptive for the present experimental family under the current encoding and solver configuration, rather than direct structural evidence for the full 128-bit specification.

In the one-key setting, the full and partial recovery percentages generally deteriorate as the parameters move toward the harder regime. Several parameter pairs still exhibit a visible gap between Full and Partial, indicating that some returned assignments remain majority-correct at the bit level even when exact recovery is no longer dominant. The quantity μ should therefore be interpreted only as a conditional diagnostic over non-timeout runs, rather than as an unconditional hardness indicator.

In the two-key setting, the additional differential constraint makes recovery easy for

Table 1: SAT summary under one-key leakage.

n	Metric	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$
16	Full	69.2	62.0	67.2	63.6	60.4
	Partial	80.8	76.0	77.6	76.0	76.4
	μ	13.5	12.9	13.3	13.0	12.9
32	Full	64.4	41.2	66.4	42.0	63.2
	Partial	77.2	90.8	80.4	93.2	81.6
	μ	26.2	26.0	26.6	26.3	26.3
48	Full	66.8	18.8	TO	TO	TO
	Partial	80.0	23.6	TO	TO	TO
	μ	40.0	38.2	TO	TO	TO
64	Full	56.8	34.0	TO	TO	TO
	Partial	74.8	92.8	TO	TO	TO
	μ	50.0	50.9	TO	TO	TO
80	Full	70.8	TO	TO	TO	TO
	Partial	84.0	TO	TO	TO	TO
	μ	68.3	TO	TO	TO	TO
96	Full	16.4	0.8	TO	TO	TO
	Partial	23.2	1.2	TO	TO	TO
	μ	73.6	80.3	TO	TO	TO
112	Full	4.8	TO	TO	TO	TO
	Partial	6.4	TO	TO	TO	TO
	μ	83.0	TO	TO	TO	TO
128	Full	0.4	TO	TO	TO	TO
	Partial	0.4	TO	TO	TO	TO
	μ	128.0	TO	TO	TO	TO

small parameter pairs, often yielding Full=Partial= 100% and $\mu = n$ in the smallest regimes. As the parameters increase, the full and partial recovery percentages generally deteriorate, and the gap between Full and Partial narrows near the onset of timeouts.

As a simple timeout-sensitivity check, for each state size n and each leakage setting for which such a threshold exists, we identified the smallest round count r for which all 250 solver runs timed out under the 600-second budget, and then reran $(n, r - 1)$ and (n, r) on 10 generated SAT instances each, using five random seeds per instance and an extended timeout of 3600 seconds. The outcomes did not qualitatively alter the main trend, suggesting that the reported behavior is not solely an artifact of the specific timeout used in the main experiments.

5.3.2 Solver statistics sanity checks

As a numerical sanity check, we inspected three quantities for signs of anomalous behavior as the parameters increase: the number of non-timeout runs, the conjunctive normal form (CNF) size, and a standard search-effort counter. For each parameter pair (n, r) , let $s_{n,r}$ denote the number of the 250 runs that returned a satisfying assignment before the 600-second timeout.

For fixed (n, r) , the SAT generator instantiates a CNF template determined by the

Table 2: SAT summary under two-key leakage.

n	Metric	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$
16	Full	100.0	100.0	100.0	100.0	100.0
	Partial	100.0	100.0	100.0	100.0	100.0
	μ	16.0	16.0	16.0	16.0	16.0
32	Full	100.0	62.8	100.0	64.8	100.0
	Partial	100.0	100.0	100.0	100.0	100.0
	μ	32.0	29.0	32.0	29.3	32.0
48	Full	100.0	89.2	TO	TO	TO
	Partial	100.0	89.2	TO	TO	TO
	μ	48.0	48.0	TO	TO	TO
64	Full	100.0	61.6	0.4	TO	TO
	Partial	100.0	100.0	0.4	TO	TO
	μ	64.0	57.8	64.0	TO	TO
80	Full	100.0	86.4	TO	TO	TO
	Partial	100.0	86.4	TO	TO	TO
	μ	80.0	80.0	TO	TO	TO
96	Full	100.0	5.2	TO	TO	TO
	Partial	100.0	12.4	TO	TO	TO
	μ	96.0	82.0	TO	TO	TO
112	Full	100.0	90.4	TO	TO	TO
	Partial	100.0	90.4	TO	TO	TO
	μ	112.0	112.0	TO	TO	TO
128	Full	100.0	TO	TO	TO	TO
	Partial	100.0	TO	TO	TO	TO
	μ	128.0	TO	TO	TO	TO

leakage setting and the SPN structure, and then substitutes the concrete leaked constants for that trial. Thus, the variable and clause counts reflect the size of the instantiated template rather than solver hardness by themselves. In the one-key setting, the generated instances follow the affine patterns $V_{n,r}^{(1)} = (8 + 39r)n/4$ and $C_{n,r}^{(1)} = (5 + 91r)n$, while in the two-key setting they follow $V_{n,r}^{(2)} = (8 + 39r)n/2$ and $C_{n,r}^{(2)} = (12 + 182r)n$, where V and C denote the numbers of variables and clauses, respectively. Hence the CNF size grows regularly with n and r , and there is no anomalous shrinkage of the instantiated SAT instances at larger r .

The main solver-side check is the behavior of $s_{n,r}$ for fixed n . In both leakage settings, $s_{n,r}$ is monotone non-increasing in r for every tested state size. For example, in the one-key setting, $s_{48,r} = (250, 78, 0, 0, 0)$ and $s_{96,r} = (77, 3, 0, 0, 0)$ for $r = 1, \dots, 5$; in the two-key setting, $s_{64,r} = (250, 250, 1, 0, 0)$ and $s_{80,r} = (250, 216, 0, 0, 0)$. Thus, once a given n enters a timeout-dominated regime, increasing r does not bring the solver back to an easier phase within the tested range.

A complementary solver-side indicator is the median number of propagations on parameter slices without censoring. In the one-key setting at $n = 16$, the clause count increases only by a factor of 4.79 from $r = 1$ to $r = 5$, whereas the median number of propagations increases from 4.67×10^4 to 7.99×10^6 , i.e., by a factor of 171. At $n = 32$, the same clause-growth factor 4.79 is accompanied by a propagation increase

from 1.84×10^6 to 2.24×10^7 , i.e., by a factor of 12.2. In the two-key setting, the analogous propagation factors are 222 at $n = 16$ and 35.7 at $n = 32$, while the clause count grows by only a factor of 4.75 in both cases. Minor local fluctuations remain, which is natural for heuristic SAT search, but we do not observe an order-of-magnitude drop at larger r that would indicate an anomalously easy regime.

These statistics do not prove hardness, but they do provide a numerical sanity check: within the tested range and under the present SAT formulation and solver configuration, increasing the SPN round count shows no evidence of a sudden recovery of solvability at larger r .

5.3.3 Weibull AFT model for censored SAT runtimes

The recovery tables above show when full or partial key recovery ceases to be practical under the imposed timeout, but they do not by themselves quantify how the SAT runtime scales once a substantial fraction of runs becomes censored. Using only solved runs would bias the analysis toward easier instances. At the same time, fitting a survival model only on the largest tested state sizes would leave too little uncensored information under the 600-second budget. For this reason, we fit a right-censored Weibull AFT model over the experimental family in (n, r) . Its role is to summarize runtime scaling under the present solver and SAT formulation, not to provide a structural model of the full 128-bit design.

Let $T_{n,r} \in (0, \infty]$ denote the time to solve for a SAT run at parameters (n, r) , and let τ be the fixed timeout. We observe $Y_{n,r} = \min(T_{n,r}, \tau)$ and treat $Y_{n,r} = \tau$ as right-censored. Separately for the one-key and two-key settings, we fit the Weibull accelerated failure time model

$$\log T_{n,r} = \mu + \gamma_n n + \gamma_r r + \sigma \varepsilon, \quad (5)$$

where ε follows the Gumbel distribution.

Under the fitted Weibull AFT model, the coefficients of both n and r are positive in the one-key and two-key settings. In the one-key setting, the estimates are $\hat{\gamma}_n = 0.1946$ and $\hat{\gamma}_r = 1.7373$; in the two-key setting, they are $\hat{\gamma}_n = 0.0946$ and $\hat{\gamma}_r = 3.0664$. Thus, within this model, larger state sizes and larger SPN round counts are associated with longer fitted time-to-solve in both leakage settings.

For a concrete queried point at $n = 128$, the fitted medians are monotone increasing over the observed range $r = 1, \dots, 5$ in both settings. In the one-key setting, the fitted median is 2.31×10^7 seconds with approximate 95% confidence interval $[1.65 \times 10^7, 3.25 \times 10^7]$ at $r = 1$, 2.41×10^{10} seconds with interval $[1.64 \times 10^{10}, 3.54 \times 10^{10}]$ at $r = 5$. In the two-key setting, the corresponding fitted medians are 2.33×10^2 seconds with interval $[1.87 \times 10^2, 2.89 \times 10^2]$ at $r = 1$, 4.94×10^7 seconds with interval $[3.78 \times 10^7, 6.45 \times 10^7]$ at $r = 5$.

The interpretation of this analysis is limited. Because the fit is based on the full experimental family, including large, heavily censored instances and reduced-size exploratory instances that do not fully preserve the 128-bit column-separation behavior of π , the predicted medians should be read as model-based summaries of the observed SAT scaling under the present encoding, solver, and timeout configuration. They are

not lower bounds on attack cost, not statements about other solver classes or attack strategies, and not stand-alone security claims for the full design.

5.4 Gröbner basis experiments

We performed a screening of direct Gröbner basis solving over $\text{GF}(2)$ for the leakage-induced polynomial systems. The systems were implemented in SageMath and solved through the Singular backend using both the standard routine (`std`) and the term-reduction variant (`slimgb`), so as to check whether simple elimination succeeds either in the default basis computation or under an implementation aimed at controlling intermediate expression growth.

The routines `std` and `slimgb` do not subsume dedicated implementations of advanced Gröbner-basis algorithms such as F4 or F5, nor related techniques such as XL. Still, if the leakage-induced systems admitted an obvious algebraic shortcut through unusually favorable elimination structure, one would expect at least some sign of it already in reduced variants under these direct computations.

Under the same instance-generation protocol as in Section 5.2, with 50 instances per parameter pair and a timeout of 600 seconds, Gröbner basis computation completed essentially only for the smallest tested setting $(n, r) = (16, 1)$; larger reduced instances were dominated by timeouts for both `std` and `slimgb`. We therefore interpret these results as an indication that the leakage-induced systems are not obviously Gröbner-friendly, rather than as an impossibility claim against stronger specialized implementations. Accordingly, the main generic-solving evaluation in this paper is based on SAT, which scales to a substantially broader range of parameters in our experiments.

5.5 Search for Exploitable Algebraic and Linear Relations

We additionally performed exploratory searches for structural properties that could plausibly lead to shortcut attacks against the leakage-induced equations. The goal of this subsection is not to prove the absence of such structure, but to check whether the induced systems exhibit obvious low-degree algebraic simplifications or unusually strong linear correlations within the tested range.

Low-degree algebraic structure. We first investigated whether the functions arising in the leakage equations exhibit unexpectedly low degree. Using higher-order derivative tests, we estimated lower bounds on the algebraic degree of the coordinate functions of

$$G(U) = U \oplus F(U) \text{ and } H_{\Delta}(U) = F(U) \oplus F(U \oplus \Delta).$$

For the one-key function G , the estimated coordinate-wise degree lower bounds are uniform across the tested state sizes and increase with the round count as 3, 9, 12, 12, 12 for $r = 1, 2, 3, 4, 5$, respectively. In particular, the lower bound already reaches 12 by $r = 3$ for every tested $n \in \{16, \dots, 128\}$.

For the two-key function H_{Δ} , we summarize the tested Δ values by reporting the minimum/median/maximum degree lower bounds over the tested choices of Δ . At the smallest state size $n = 16$, these bounds increase from $(1, 2, 2)$ at $r = 1$ to $(8, 8, 8)$

at $r = 2$, and then to $(12, 12, 12)$ for $r \geq 3$. More generally, across the tested range, the best reported lower bound reaches 12 for all n once $r \geq 3$, while the median lower bound is 12 for $n \in \{16, 48, 80, 112\}$ and 6 for $n \in \{32, 64, 96, 128\}$.

Thus, although the two-key setting is weaker at small rounds, the estimated degree lower bounds still increase with the number of rounds and do not exhibit sudden drops. This behavior suggests that the trends observed in Section 5.3 are not explained by an low-degree algebraic simplification.

Sampling-based scan for linear correlations. We also performed a sampling-based scan for linear correlations in the same functions. Random input masks were combined with low-weight output masks (Hamming weight 1–2), and the empirical bias of the resulting linear approximations was estimated using 2^{14} input samples per mask. For each parameter pair (n, r) , we tested 16,384 mask pairs and recorded the maximum observed bias, together with a finite-sample baseline obtained from random Boolean functions under the same sampling conditions.

For the one-key function $G(U) = U \oplus F(U)$, the maximum observed bias over the tested parameter range lies between 0.01434 and 0.03796, while the corresponding baseline lies between 0.01398 and 0.01880. The largest observed value occurs at the smallest tested setting $(n, r) = (16, 1)$, where the maximum bias is 0.03796 versus a baseline of 0.01538. Excluding this smallest instance, the observed maxima lie in the range 0.01434–0.02075, and the largest absolute excess over baseline is 0.00330 (at $(n, r) = (64, 1)$).

For the two-key function $H_\Delta(U) = F(U) \oplus F(U \oplus \Delta)$, whose behavior depends on Δ , we report for each (n, r) the worst observed maximum bias over the tested values of Δ . These worst-case observed maxima lie between 0.01453 and 0.24597, whereas the corresponding baselines lie between 0.01453 and 0.01862. The only conspicuous outlier is again the smallest tested setting $(n, r) = (16, 1)$, for which the worst-case maximum bias is 0.24597 versus a baseline of 0.01770. Excluding this case, the worst-case observed maxima lie in the range 0.01453–0.02148, and the largest absolute excess over baseline is 0.00427 (at $(n, r) = (16, 2)$). Taken together with the degree results above, these data do not indicate simple algebraic or linear shortcuts within the tested range. The most noticeable departures from the finite-sample baseline occur only in the smallest round regime, especially in the two-key setting at $(n, r) = (16, 1)$; outside this weakest regime, the observed maximum biases remain close to baseline. In particular, at $n = 128$, the observed maxima stay in the range 0.0150–0.0161 in the one-key setting and 0.0153–0.0176 in the two-key setting, remaining close to the corresponding finite-sample baselines across all tested r .

6 Conclusion

We studied whether the effect of boundary round-key leakage can be addressed directly at the level of key-schedule structure, without delegating the main security argument to an auxiliary keyed primitive such as a PRF, through the explicit nonlinear construction $\text{RK}_i = K \oplus F(K \oplus T(i))$, where $T(i)$ is public and F is a public SPN-based permutation.

Under the boundary-leakage model, leakage of one end round key reduces to an equation of the form $Z = U \oplus F(U)$, whereas leakage of both end round keys yields the corresponding two boundary instances determined by the first and last round indices. Within the tested parameter range, the smallest round instantiations appear weakest in the two-key setting, whereas larger SPN round counts are associated with greater difficulty for the generic techniques considered here. The AFT analysis should be read in the same restricted sense: it summarizes the runtime scaling of the present SAT formulation over the experimental family, rather than providing a stand-alone structural guarantee for the full 128-bit specification.

Accordingly, these results do not constitute a proof of security or a parameter-independent claim about all choices of N_F ; they indicate only that, for the tested settings, larger SPN round counts are associated with greater difficulty of solving the induced leakage equations.

Acknowledgments

This work was supported by the Japan Society for the Promotion of Science KAKENHI Grant Number 20K11817.

References

- [1] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1996.
- [2] National Institute of Standards and Technology, “Data Encryption Standard (DES).” FIPS PUB 46-3, 1999.
- [3] P. Kocher, J. Jaffe, and B. Jun, “Differential power analysis,” in *Proceedings of the 19th Annual International Cryptology Conference on Advances in Cryptology (CRYPTO’99)*, pp. 388–397, 1999.
- [4] E. Brier, C. Clavier, and F. Olivier, “Correlation power analysis with a leakage model,” in *Proceedings of 6th International Workshop on Cryptographic Hardware and Embedded Systems (CHES2004)*, vol. 3156 of *Lecture Notes in Computer Science*, pp. 16–29, 2004.
- [5] T. S. Messerges, “Using second-order power analysis to attack dpa-resistant software,” in *Proceedings of 2nd International Workshop on Cryptographic Hardware and Embedded Systems (CHES2000)*, vol. 1965 of *Lecture Notes in Computer Science*, pp. 238–251, 2000.
- [6] E. Biham and A. Shamir, “Differential fault analysis of secret key cryptosystems,” in *Proceedings of the 17th Annual International Cryptology Conference on Advances in Cryptology (CRYPTO’97)*, vol. 1294 of *Lecture Notes in Computer Science*, pp. 513–525, 1997.

- [7] G. Piret and J.-J. Quisquater, “A differential fault attack technique against spn structures, with application to the aes and khazad,” in *Proceedings of 5th International Workshop on Cryptographic Hardware and Embedded Systems (CHES2003)*, vol. 2779 of *Lecture Notes in Computer Science*, pp. 77–88, 2003.
- [8] P. Dusart, G. Letourneux, and O. Vivolo, “Differential fault analysis on aes,” in *Proceedings of 1st International Conference on Applied Cryptography and Network Security (ACNS2003)*, vol. 2846 of *Lecture Notes in Computer Science*, pp. 293–306, 2003.
- [9] J. A. Halderman, S. D. Schoen, N. Heninger, W. Clarkson, W. Paul, J. A. Candalrino, A. J. Feldman, J. Appelbaum, and E. W. Felten, “Lest we remember: cold-boot attacks on encryption keys,” *Communications of the ACM*, vol. 52, no. 5, p. 91–98, 2009.
- [10] P. Joshi and B. Mazumdar, “Deep round key recovery attacks and countermeasure in persistent fault model: a case study on GIFT and KLEIN,” *Journal of Cryptographic Engineering*, vol. 14, no. 1, pp. 103–125, 2024.
- [11] J. Kelsey, B. Schneier, and D. Wagner, “Key-schedule cryptanalysis of IDEA, G-DES, GOST, SAFER, and triple-DES,” in *Proceedings of the 16th Annual International Cryptology Conference on Advances in Cryptology (CRYPTO’96)*, *Lecture Notes in Computer Science*, pp. 237–251, 1996.
- [12] H. Yoshikawa, M. Kaminaga, and A. Shikoda, “Round addition using faults for generalized feistel network,” *IEICE Transactions on Information and Systems*, vol. E96-D, no. 1, pp. 146–150, 2013.
- [13] R. Ueno, N. Homma, and T. Aoki, “Efficient DFA on SPN-Based Block Ciphers and Its Application to the LED Block Cipher,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E98-A, no. 1, pp. 182–191, 2015.
- [14] G. V. Bard, *Algebraic Cryptanalysis*. New York, NY: Springer, August 2009.
- [15] A. Biryukov and D. Wagner, “Slide attacks,” in *Proceedings of 6th International Workshop on Fast Software Encryption (FSE’99)*, vol. 1636 of *Lecture Notes in Computer Science*, pp. 245–259, 1999.
- [16] A. Biryukov and D. Khovratovich, “Related-key cryptanalysis of the full AES-192 and AES-256,” in *Proceedings of 15th International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT 2009)*, vol. 5912 of *Lecture Notes in Computer Science*, pp. 1–18, 2009.
- [17] Chien-Ning Chen and Sung-Ming Yen, “Differential fault analysis on AES key schedule and some countermeasures,” in *Proceedings of 8th Australasian Conference on Information Security and Privacy (ACISP’03)*, vol. 2727 of *Lecture Notes in Computer Science*, pp. 118–129, 2003.

- [18] C. Clavier, D. Marion, and A. Wurcker, “Simple power analysis on AES key expansion revisited,” in *Proceedings of 16th International Workshop on Cryptographic Hardware and Embedded Systems (CHES2014)*, vol. 8731 of *Lecture Notes in Computer Science*, pp. 279–297, 2014.
- [19] J. Choy, A. Zhang, K. Khoo, M. Henricksen, and A. Poschmann, “Aes variants secure against related-key differential and boomerang attacks,” in *Proceedings of 5th IFIP WG 11.2 International Workshop on Information Security Theory and Practice: Security and Privacy of Mobile Devices in Wireless Communication (WISTP2011)*, vol. 6633 of *Lecture Notes in Computer Science*, pp. 191–207, 2011.
- [20] J. Nechvatal, E. Barker, L. Dodson, M. Dworkin, J. Foti, and E. Roback, “Status report on the first round of the development of the advanced encryption standard.” National Institute of Standards and Technology, 1999.
- [21] A. Bogdanov *et al.*, “Present: An ultra-lightweight block cipher,” in *Proceedings of 9th International Workshop on Cryptographic Hardware and Embedded Systems (CHES2007)*, vol. 4727 of *Lecture Notes in Computer Science*, pp. 450–466, 2007.